

Sew and Code! Understanding How to Construct Bezier Curves

Brittany Morago
University of North Carolina Wilmington
Wilmington, USA
moragob@uncw.edu

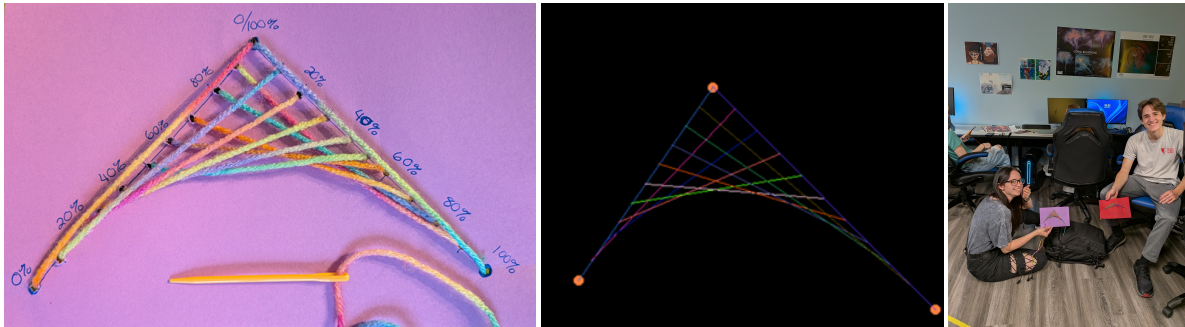


Figure 1: Left to right: Sewn Bezier curve, programmed curve, Computer Graphics students sewing in UNCW Digital Art's lab.

Abstract

Sew and Code is an assignment that combines art, math, and programming for undergraduate Computer Graphics classes. It builds on String Art techniques and has students directly relate the measurements of their sewing to the variables and structure of their code as they implement the DeCasteljau Algorithm to create Bezier curves. Students switch back and forth between sewing and programming, building up similar visual results on paper and the screen simultaneously, motivating the study of Bezier curves.

CCS Concepts

- Applied computing → Interactive learning environments;
- Computing methodologies → Computer graphics; Shape modeling.

Keywords

Interactive assignments, computer graphics, curves

ACM Reference Format:

Brittany Morago. 2026. Sew and Code! Understanding How to Construct Bezier Curves. In *Special Interest Group on Computer Graphics and Interactive Techniques Conference Educator's Forum (SIGGRAPH Educator's Forum '26)*, July 19–23, 2026, Los Angeles, CA, USA. ACM, New York, NY, USA, 3 pages. <https://doi.org/10.1145/3799829.3812513>

1 Introduction

Undergraduate Computer Graphics classes at their very heart fuse programming, math, algorithms, art, and design and draw in students interested in any of these areas. Reaching groups of students

Summary	Covers constructing Bezier curves by cycling between sewing a step of the DeCasteljau algorithm and programming the same step.
Learning Outcomes	Understand properties of Bezier curves and implement DeCasteljau algorithm
Classification(s)	Fundamentals, Image Manipulation
Audience	Computer Science and Digital Arts juniors
Dependencies	2 semesters of programming (any language)
Prerequisites	1+ C++ programming assignments
Strengths	Link to art and non-programming tasks
Weaknesses	DeCasteljau is a tricky algorithm to implement, especially for those with little love of CS
Variants	Make an interactive version where CP's can be moved and curve is altered in real-time
Assessment	Each "stage" is assigned points and can be graded on completion

Table 1: Sew and Code Metadata

whose prior efforts lie in only one or two of these areas and providing the scaffolding for all of them to implement a visually interactive program of a mathematical concept can be a pedagogical challenge. The Sew and Code assignment is designed to accomplish this as students learn how to construct Bezier curves using the DeCasteljau Algorithm, a classic topic in many CG classes [Shirley et al. 2009]. Students cycle between sewing a step of the DeCasteljau algorithm, programming the same step, and seeing similar results on paper and on the screen before moving to the next step. This makes use of classical String Art activities [von Renesse and Ecke 2016] but has students dig deeper into the exact math and measurements of the sewing portion. In their coded implementation, variable names and programming structures are directly linked to the sewing process.

I have developed this assignment over several years of teaching CSC 370 Computer Graphics, a course taken by Computer Science and Digital Arts majors. Students take at least 2 semesters of foundational Computer Science/programming classes and 2 semesters



This work is licensed under a Creative Commons Attribution 4.0 International License. *SIGGRAPH Educator's Forum '26*, Los Angeles, CA, USA
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2552-4/26/07
<https://doi.org/10.1145/3799829.3812513>

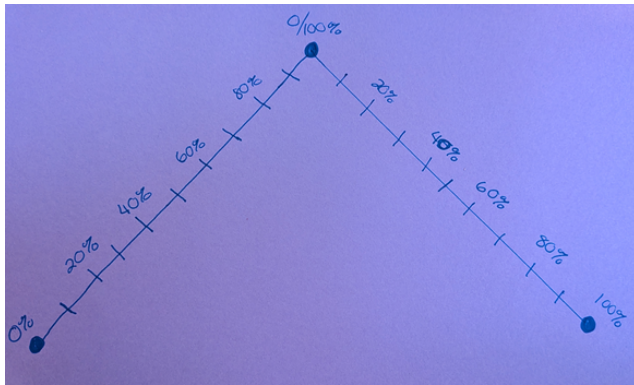


Figure 2: Stage 1 guiding points for sewing (final sewn curve shown in Fig. 1).

of Calculus as pre-requisites for Computer Graphics. Given the Digital Arts component, many students who take this class have extensive experience with analog art but are still developing their programming skills. Sew and Code is designed to motivate these students by showing the overlaps between math, art, and CS and relating to their established talents.

2 Bezier Overview and Terminology

Bezier curves are used in many areas of computer graphics including guiding object paths and speeds for animation, modeling organic surfaces, creating fonts, etc. The DeCasteljau Algorithm is a recursive set of steps that can be followed to construct Bezier curves through drawing, sewing, and programming. We will use the following terms in describing Sew and Code:

- **Control points (CP):** The user clicks **control points** on the screen (or marks on paper) to guide the curve's shape.
- **Curve order:** The underlying equation representing a Bezier curve is of **order 1** less than the number of guiding **CP's**. A second order quadratic curve will have 3 **CP's**, a third order cubic curve will have 4 **CP's**, etc.
- **Steps:** The curve is gradually built-up taking certain percentage **steps** from 0% to 100%. The **resolution** of the final Bezier curve is the number of **steps** taken.
- **Legs/Levels:** We start by taking a defined size **step** between the first and second **CP's** and drawing a **leg** between these points. We repeat this, taking the same size step between the second and third **CP's** and so on until we hit the last 2 **CP's**, creating 1 less **leg** than the number of **CP's**. These legs create one **level**. If more than 1 **leg** is in a **level**, we repeat this process along consecutive **legs**, having 1 less **leg** with each additional **level**.

3 Assignment Steps

The Sew and Code assignment has students document their process as they follow along with the following main steps:

Sewing Stage 1: Students sew the legs of a quadratic curve (3 **CP's**) and mark percentage steps along the way (Fig. 2).

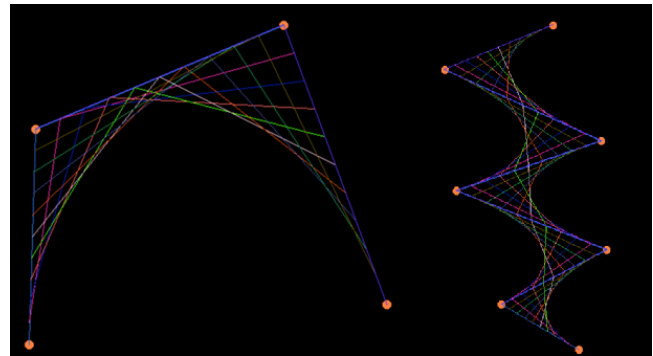


Figure 3: Coding Stage 2 output for 2 different sets of control points with each step shown in a different random color.

Coding Stage 1: Then they write code to display the same shape matching their program structure and variables to the stitching they just completed (Fig. 1) such as:

*"Set up a loop that goes through the resolution of the curve you want to draw. Start by making this match the number of **stitches** you sewed. If you did a stitch for legs at 10% steps, you have stitches for: 0%, 10%, 20%, 30%, 40%, 50%, 60%, 70%, 80%, 90%, and 100%, or 11 stitches total. In this case, your loop should take steps of 0.1 to start so the resolution loop goes from 0.0 to 1.0. Call the counter variable **step**."*

Students are then guided to write linear interpolation equations ($newPt = (1 - step) * leftPt + step * rightPt$) inside this loop to find: a. The point that is **step** percent between the first 2 **CP's** and b. The point that is **step** percent between the second 2 **CP's**. They draw a line between these two points using a random color.

Coding Stage 2: The next stage is generalizing Stage 1 for any number of control points while still only displaying one level of DeCasteljau (Fig. 3): "1) You'll still get control points from user interaction (pts) but now feel free to click more than 3 points before closing the window. 2) Inside your resolution loop: i. Generate a random RGB color ii. Make new empty vectors to store the points produced by your current iteration of DeCasteljau (call these newPts) iii. Loop through all the segments connecting consecutive pairs of pts and add a pt to newPts based on your resolution. (You wrote this part out in Stage 1 with the linear interpolation formula. Now you're just putting that block in a loop and saving the coordinates in newPts.) iv. Loop through all the segments connecting consecutive pairs of newPts and draw a line for each segment."

Sewing Stage 2: Students step back from coding and sew another cubic Bezier curve with 4 **CP's**, requiring 2 levels. Students are instructed to take larger size steps (25-33%). Otherwise, the number of overlapping stitches can become cumbersome. Students sew the 2 levels in 2 different colors (Fig. 4).

Coding Stage 3: The output looks like Sewing Stage 2 (Fig. 4). It adds onto the resolution loop. At the beginning of each iteration, students restart with the original **CP's** the user clicked (putting these in lists called currentPts), generate one level's worth of points using interpolation, and then use these new points to guide the next level (swapping newPts and currentPts). They write a loop around their existing code for the number of levels needed for DeCasteljau.

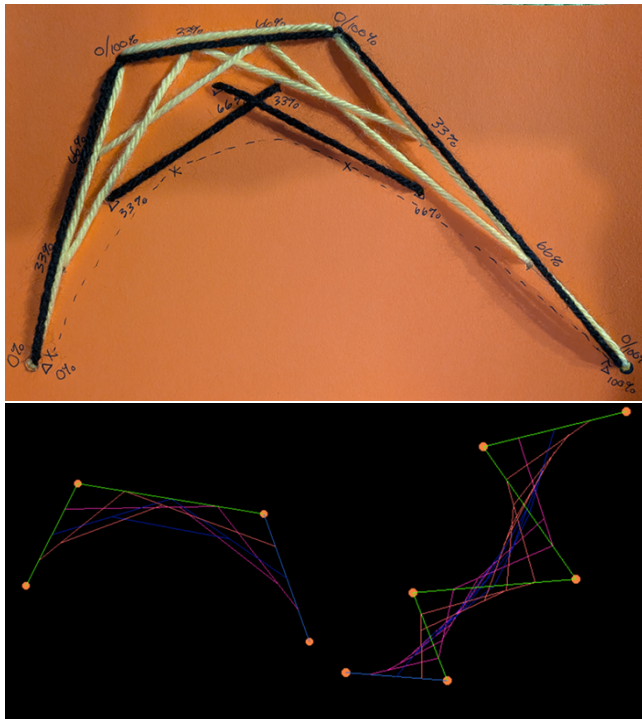


Figure 4: Sewing Stage 2 (Top) and Coding Stage 3 (Bottom Left) produce similar output with 4 CP. Coding Stage 3 with 6 CPs (Bottom Right).

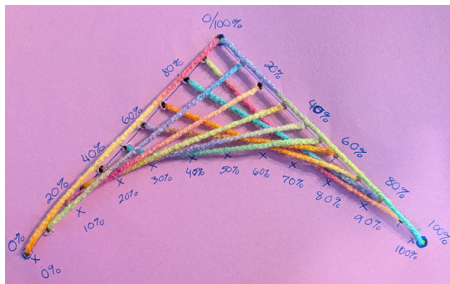


Figure 5: Sewing Stage 3 result of marking points of the final Bezier curve along the stitched legs.

If the user clicks 3 points, they need 1 level of DeCasteljau. If the user clicks 4 points, they need 2 levels, etc.

Sewing Stage 3: Students draw several of the points on the final curve along their sewn legs (Fig. 5). For instance, on the 3 point curve with 10% steps, they will draw a dot 10% along the 10% leg, 20% along the 20% leg, etc.

Coding Stage 4: Students draw those same dots in their code (Fig. 6). They run their levels loop one additional iteration. When the levels loop is complete, they draw a circle at the first coordinate stored in their currentPts list.

Coding Stage 5: At this final stage, students simply comment out the code that shows all in the levels and legs involving in constructing the curve and leave just the final points (Fig. 6).

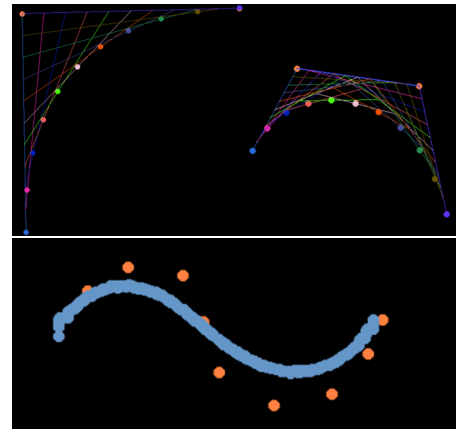


Figure 6: Top: Stage 4's output with 10% steps on 2 different control point sets. Bottom: Final curve. CPs in orange and final curve points in blue.

4 Bonus

The structure of this assignment is such that the user clicks CP's and a static curve is displayed. If the user wants to change something they must restart the program. Students looking for an extra feature to add on could adapt the code so that the final display is interactive, users can update CP locations, and a new curve is computed and displayed.

5 Evaluating Student Learning and Conclusion

This assignment began in a format where I covered the underlying properties of Bezier curves, the steps of DeCasteljau, and asked students to implement the algorithm's steps using a starter program. The starter code provided the framework for displaying a window, storing the coordinates of user clicked points, and displaying a final image. It was common for my students to struggle with the assignment as they had difficulty understanding the steps of DeCasteljau and how they all work together. I then added in the assignment component of having students sew 2nd order and 3rd order curves before implementing the algorithm. After seeing increased student success, I expanded it so that students move back and forth between sewing and coding and see the same intermediate results before tackling the next step. With each version of this assignment I have seen student completion rates increase.

Acknowledgments

To my UNCW CSC 370 Computer Graphics students.

References

- Peter Shirley, Michael Ashikhmin, and Steve Marschner. 2009. *Fundamentals of computer graphics*. AK Peters/CRC Press, 359–404.
- Christine von Renesse and Volker Eke. 2016. Discovering the art of mathematics: Using string art to investigate calculus. *PRIMUS* 26, 4 (2016), 283–296.